

TPMS

胎壓檢測器自動抓取系統

技術文件與流程說明

版本：v2.0（含進度追蹤與週期檢查功能）

檔案位置：z:\221\胎壓檢測器自動抓取.py

文件日期：2026年7月16日

目錄

1. 系統概述
2. 檔案結構說明
3. 原始程式碼問題分析
4. 檢查邏輯詳細說明
5. 週期檢查機制 (v2.0 新增)
6. 完整流程圖
7. 改進項目總結
8. 附錄

1. 系統概述

本系統用於從德國汽車零件網站 (interpneu-raederkonfigurator.de) 自動抓取 TPMS (胎壓偵測器) 感測器資料，並儲存至本地 SQLite 資料庫，同時匯出 Excel 報表。

主要功能

- 自動抓取所有品牌、車系、車型的 TPMS 感測器資料
- 支援斷點續傳 (7天內從上次中斷點繼續)
- 每7天自動全面檢查更新
- 只檢查最近2年內的車款 (效能優化)
- 自動匯出 SQLite 備份檔與 Excel 報表

資料欄位

品牌、車系、型號、年份起點、年份終點、HSN、TSN、建設日期(Baujahr)、OE感測器、廠商(Hersteller)、頻率(Frequenz)

2. 檔案結構說明

程式執行後會產生以下檔案：

檔案名稱	說明
胎壓檢測器自動抓取.py	主程式碼
胎壓檢測器.db	SQLite 資料庫
胎壓檢測器.sql	資料庫純文字備份
胎壓檢測器/{品牌}_Data.xlsx	各品牌 Excel 報表

3. 原始程式碼問題分析

問題一：靜默吞錯誤

```
except:  
    return set()
```

問題：會隱藏所有 bug，難以除錯

建議：改用 `except Exception as e:` 並印出錯誤訊息

問題二：年份更新判斷缺陷

原始邏輯只比對「品牌+車系+型號+年份起點」四個欄位。

若網站更新了某年份的感測器資料（如 OE 編號、廠商、頻率），但年份沒變，系統會認為「已存在」而跳過。

問題三：中斷後從頭開始

原始程式每次啟動都從第一個品牌開始，逐一比對資料庫。

若中斷後重啟，會重複比對已完成的品牌，浪費時間。

問題四：舊款車重複檢查

舊款車型（如5年前的車款）通常不會更新感測器資料，但原始程式仍會逐一檢查，造成不必要的 API 呼叫。

4. 檢查邏輯詳細說明

4.1 已存在判斷

系統在啟動時，會從資料庫載入所有已抓取的記錄：

`model_identity` = (品牌, 車系, 型號, 年份起點)

若此 tuple 已存在於記憶體中，則視為「已抓取」。

4.2 計數器邏輯 (already_exists_streak)

versions 依年份從新到舊排序後：

`idx=0` (2024) -> 已存在 -> 強制檢查, `streak=0`

`idx=1` (2023) -> 已存在 -> `streak=1`

`idx=2` (2022) -> 已存在 -> `streak=2`

`idx=3` (2021) -> 已存在 -> `streak=3`

`idx=4` (2020) -> 已存在 -> `streak=4`

`idx=5` (2019) -> 已存在 -> `streak=5` -> break 跳過

只有連續5個已存在的舊年份才會中斷，跳過後續更舊的車款。

若車款只有1~3筆，不會觸發 break，正常處理完畢。

4.3 強制檢查機制

各型號的最新年份 (`idx=0`) 無論如何都會強制檢查，
確保最新的年份終點與感測器資料無更新。

4.4 感測器內容比對 (v2.0 新增)

即使車款已存在，也會比對感測器內容是否有更新：

- OE 感測器編號
- 廠商 (Hersteller)
- 頻率 (Frequenz)

若任一欄位有變動，則更新資料庫記錄。

這解決了「年份不變但感測器資料更新」的問題。

5. 週期檢查機制 (v2.0 新增)

5.1 兩種執行模式

模式一：全面檢查模式

- 觸發條件：距上次執行超過7天，或首次執行
- 行為：從第一個品牌開始，全部重新檢查
- 適用場景：每月定期更新、手動觸發全面檢查

模式二：繼續進度模式

- 觸發條件：7天內重新執行
- 行為：從上次中斷點繼續，跳過已完成的品牌
- 適用場景：程式中斷後重啟

5.2 進度追蹤表 (scrape_progress)

新增資料表結構：

id (主鍵)
last_brand (上次處理的品牌)
last_car_class (上次處理的車系)
last_type_group (上次處理的型號群組)
last_version_idx (上次處理的版本索引)
mode (執行模式)
created_at (建立時間)
last_updated (最後更新時間)

5.3 只檢查最近2年

needs_check(year_from, year_to) 函式：

- 計算車款的結束年份
- 若結束年份 $\geq$ 當前年份-2，則需要檢查
- 否則直接跳過，不呼叫 API

效果：大幅減少不必要的 API 呼叫，提升執行效率。

5.4 流程範例

情境一：首次執行

- > 沒有進度記錄 -> 進入全面檢查模式
- > 從第一個品牌開始抓取

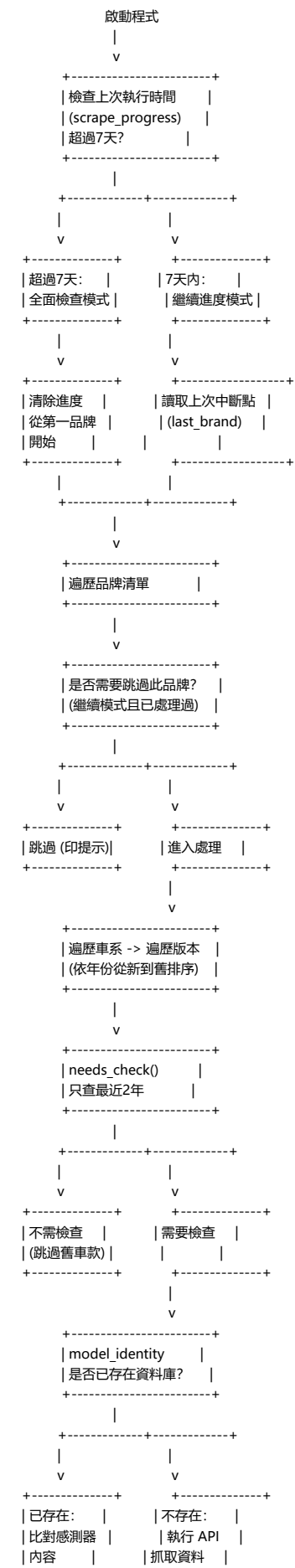
情境二：程式中斷後重啟 (7天內)

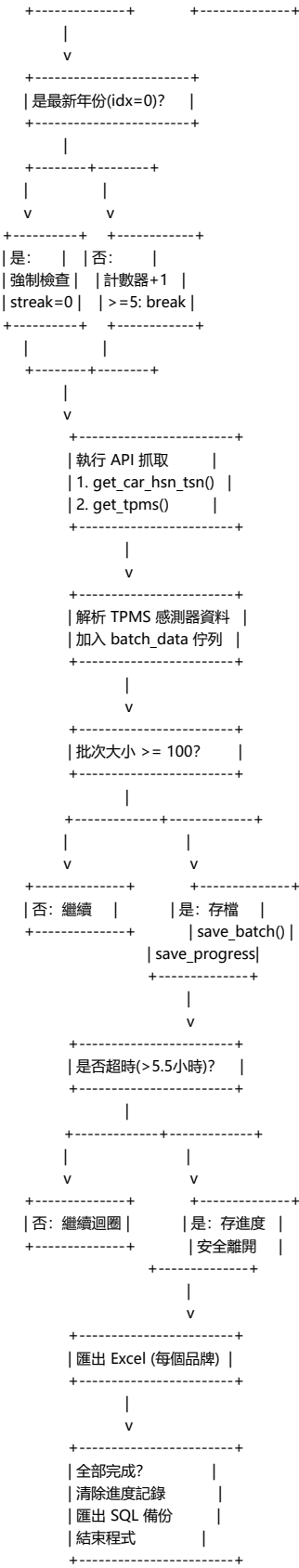
- > 讀取進度 -> 從上次中斷的品牌繼續
- > 跳過已完成的品牌

情境三：下個月執行

- > 距上次超過7天 -> 進入全面檢查模式
- > 全部重新檢查，確保資料最新

6. 完整流程圖





7. 改進項目總結

v2.0 主要改進

改進項目	說明
進度追蹤	新增 scrape_progress 記錄表，儲存上次中斷點
7天分隔	超過7天自動全面檢查，7天內從斷點繼續
只查2年	needs_check() 判斷，舊車款直接跳過
感測器比對	已存在的車款也比對內容是否有更新
中斷恢復	超時時自動存進度，下次從斷點恢復
錯誤處理	將靜默 except 改為可見的錯誤訊息

使用建議

- 首次執行：自動進入全面檢查模式
- 中斷重啟（7天內）：自動從斷點繼續
- 每月更新：超過7天後自動全面檢查
- 手動全面檢查：刪除胎壓檢測器.db 後重新執行

8. 附錄

檔案位置

程式碼：z:\221\胎壓檢測器自動抓取.py

資料庫：z:\221\胎壓檢測器\胎壓檢測器.db

SQL 備份：z:\221\胎壓檢測器\胎壓檢測器.sql

Excel 報表：z:\221\胎壓檢測器\{品牌名稱}_Data.xlsx

執行方式

```
# 基本執行
python "z:\221\胎壓檢測器自動抓取.py"

# 強制全面檢查（刪除資料庫）
del "z:\221\胎壓檢測器\胎壓檢測器.db"
python "z:\221\胎壓檢測器自動抓取.py"
```

API 端點說明

資料來源：interpneu-raederkonfigurator.de

API 端點：

/api/cars/manufacturers - 取得品牌清單

/api/cars/classes - 取得車系清單

/api/cars/type-groups - 取得型號群組

/api/cars/version-groups - 取得版本清單

/api/cars/car - 取得車輛詳細資料 (HSN/TSN)

/api/tpms/carTpms - 取得 TPMS 感測器資料